

3. 无重复字符的最大子串

全部提交记录

通过 988 / 989 个通过的测试用例
白日梦家 提交于 2026.03.13 22:13

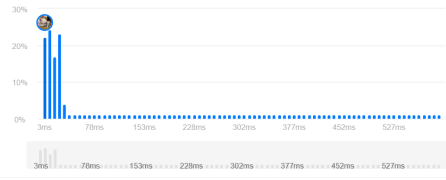
面向在校学生的专享特惠
完成认证享 7 折 Plus 会员，享更多学业及职业成长帮助

执行用时分布

7 ms | 击败 77.79%

消耗内存分布

11.66 MB | 击败 64.25%



3ms 7ms 153ms 228ms 302ms 377ms 452ms 527ms

代码 C++

```
1 class Solution {
2 public:
3     int lengthOfLongestSubstring(string s) {
4         int left = 0;
5         int maxlen = 0;
6         unordered_map<char, int> appeared;
7
8         for (int right = 0; right < s.length(); ++right) {
```

C++ 智能模式

```
1 class Solution {
2 public:
3     int lengthOfLongestSubstring(string s) {
4         int left = 0;
5         int maxlen = 0;
6         unordered_map<char, int> appeared;
7
8         for (int right = 0; right < s.length(); ++right) {
9             char cur = s[right];
10            if (appeared.find(cur) != appeared.end() && appeared[cur] >= left) {
11                left = appeared[cur] + 1;
12            }
13            appeared[cur] = right;
14
15            int curLen = right - left + 1;
16            maxlen = max(maxlen, curLen);
17        }
18        return maxlen;
19    }
20 };
```

行 14, 列 1 | 已存储

运行 提交

测试用例 测试结果

```
1 int lengthOfLongestSubstring(string s) {
2     int left = 0;           // 滑动窗口左边界
3     int maxlen = 0;         // 记录最长无重复子串长度
4     unordered_map<char, int> appeared; // 记录每个字符最近一次出现的下标
5
6     for (int right = 0; right < s.length(); ++right) {
7         char cur = s[right];
8
9         // 如果当前字符已出现过，且其上次位置在当前窗口内，说明出现重复
10        if (appeared.find(cur) != appeared.end() && appeared[cur] >= left) {
11            left = appeared[cur] + 1; // 移动左边界到重复字符的下一个位置
12        }
13
14        appeared[cur] = right; // 更新当前字符的最新位置
15
16        // 更新最长长度：当前窗口 [left, right] 的大小
17        maxlen = max(maxlen, right - left + 1);
18    }
19
20    return maxlen;
21 }
```

1385. 两个数组间的距离值

全部提交记录

通过 101 / 103 个通过的测试用例
白日梦家 提交于 2026-03-13 22:43

分析 写题解

百度面试突击手册
众里寻你恰百度

→

执行用时分布
3 ms 击败 48.62%

消耗内存分布
16.33 MB 击败 60.99%



Bar chart showing performance distribution. The x-axis represents execution time in milliseconds (2ms to 18ms) and memory usage in MB (2MB to 18MB). The y-axis represents the percentage of solutions (0% to 75%). The chart shows that the solution is faster than approximately 48.62% of other solutions and uses less memory than approximately 60.99% of other solutions.

代码 C++

```
1 class Solution {
2 public:
3     int findTheDistanceValue(vector<int>& arr1, vector<int>& arr2, int d) {
4         sort(arr2.begin(), arr2.end());
5         int count = 0;
6
7         for (int num : arr1) {
8             auto pos = lower_bound(arr2.begin(), arr2.end(), num - d);
9             if (pos == arr2.end() || *pos > num + d) {
10                 count++;
11             }
12         }
13         return count;
14     }
15 };
```

行 15, 列 3 已存储 运行 提交

代码 两种方法: 二分查找

```
1 class Solution {
2 public:
3     int findTheDistanceValue(vector<int>& arr1, vector<int>& arr2, int d) {
4         sort(arr2.begin(), arr2.end());
5         int count = 0;
6
7         for (int num : arr1) {
8             auto pos = lower_bound(arr2.begin(), arr2.end(), num - d);
9             if (pos == arr2.end() || *pos > num + d) {
10                 count++;
11             }
12         }
13         return count;
14     }
15 };
```

行 15, 列 3 已存储 运行 提交

```
1 class Solution {
2 public:
3     int findTheDistanceValue(vector<int>& arr1, vector<int>& arr2, int d) {
4         sort(arr2.begin(), arr2.end()); // 对arr2排序
5         int count = 0; // 统计满足条件的元素数量
6
7         for (int num : arr1) {
8             // 查找 arr2 中第一个不小于 (num - d) 的元素
9             auto pos = lower_bound(arr2.begin(), arr2.end(), num - d);
10
11             // 若该位置越界, 或所指元素大于 (num + d), 说明 [num-d, num+d] 区间内无
arr2 元素
12             if (pos == arr2.end() || *pos > num + d) {
13                 count++;
14             }
15         }
16         return count;
17     }
18 };
```